Technical Documentation: Hospital Management System

1. Installation and System Architecture

a. Instructions

- Copy the application files to the web server root (e.g., /var/www/html or your local htdocs folder).
- Open Includes/config.php and verify the database hostname.
- Upload hospital_database.sql from Database folder to your database system to reconstruct the data.
- Access the application via the browser at the configured local address (e.g., http://localhost:80).

b. Overview

The app is a web-based end-to-end Hospital Management System designed to allow QMC doctors and administrators to access patient data. Although the database is currently small, the system is designed to handle data scaling and new features. The system is built on the LAMP stack.

c. Infrastructure

The system comprises three interconnected services communicated over an internal Docker network *(Figure 1)*, exposing specific ports for user and administrative access.

- User access — port 80: the PHP-Apache container handles HTTP traffic, processing inputs and rendering HTML.
- Database — internal: the PHP application communicates via MySQL protocol on internal port 3306.
- Database administration — port 8000: A phpMyAdmin container provides database management.

d. File Organisation

The project follows a modular file structure mapped to the web server's document root (/var/ww/html inside the container):

- Root directory: contains the main application pages (e.g. prescribe.php, add_doctor.php, add_test.php). It also includes a generic error_page and .htaccess.

- Includes folder: this directory contains reusable components and configuration files.
    - config.php: manages the database connection settings.
    - header.php/footer.php/left_menu: Ensures a consistent UI layout across the platform.
    - loginaction.php: contains login instructions.
    - audit_trail.php: includes helper functions that are called on each page where CRUD actions might be executed to record all the activity for audit purposes.
- Css folder with style pages.

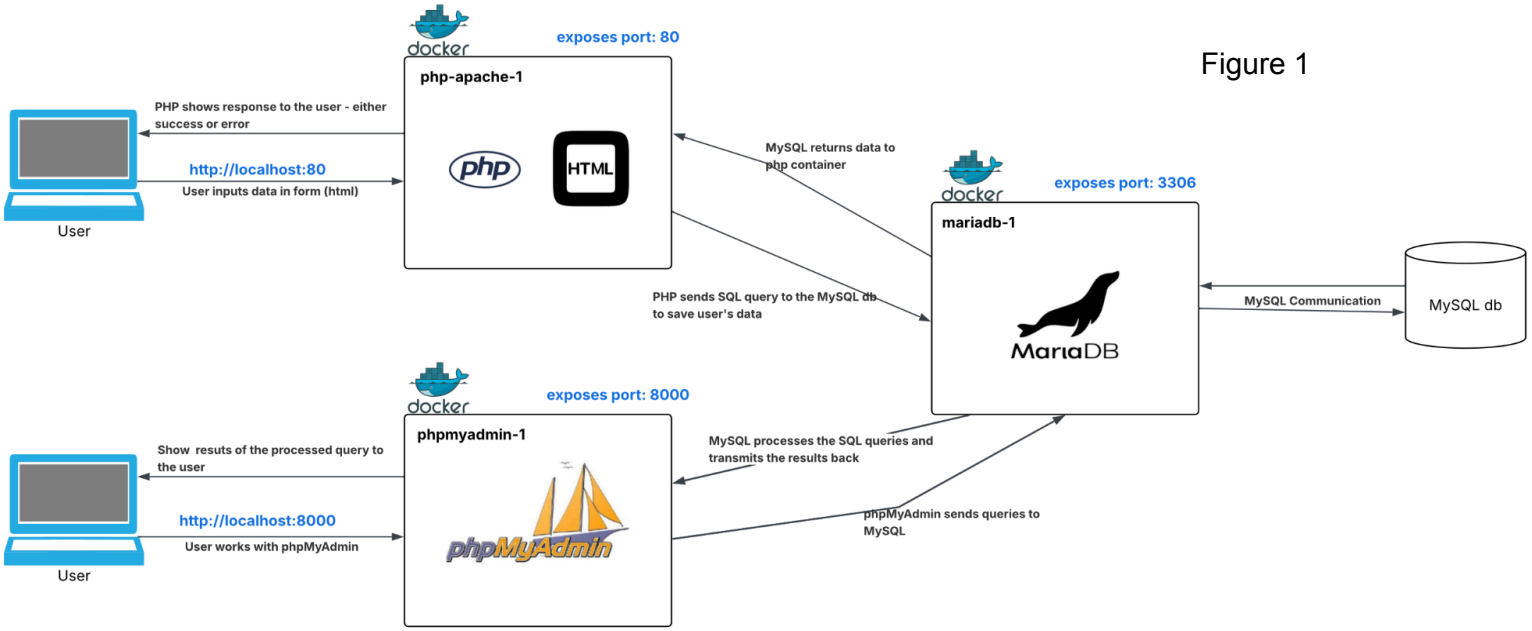

Figure 1

2. Design Decisions

a. Database Structure

The schema follows Third Normal Form (3NF). This is partially achieved through putting categorical attributes such as gender, consultant status, and department names into dedicated lookup tables (gender, consultant_status, department), which are referenced via foreign keys rather than repetitive text strings. *(Figure 2).*

- Some of the design decisions in detail
    - Parking module cardinality:
        - One request per doctor: the parking_request table uses doctorid as its Primary Key and Foreign Key. This constraint makes it impossible for a single doctor to have more than one active request at a time.
        - Workflow: the relationships enforce a linear workflow: Doctor -> Request -> Permit.

- Approval Link: The parking_permit table links back to the request, ensuring that a permit cannot exist without a preceding, approved request.

- Integrity constraints:
  - ON UPDATE CASCADE: used for linked IDs. If a Doctor's staffno is updated, the change automatically ripples through to child tables like patient_test.
  - ON DELETE RESTRICT: used to restrict certain deletions, e.g. test cannot be deleted if it is prescribed to a patient.

b. Styling Approach

Frontend styling uses a layered approach: bootstrap 5 for layout, overridden by a custom “global” CSS file for branding, with minimal in-line CSS for page-specific needs.

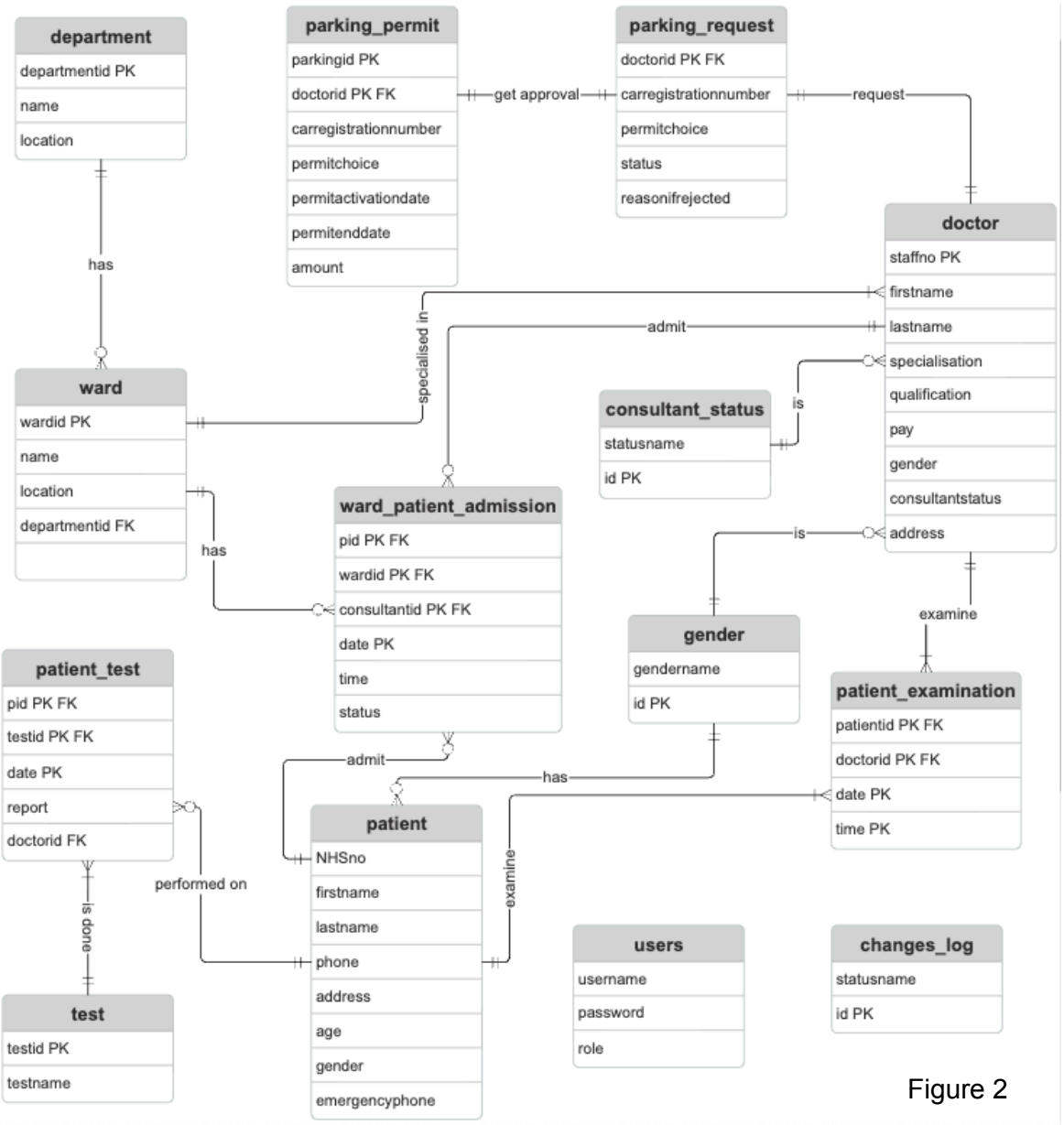

Figure 2

3. Key Data Flows

a. PHP Pages Logic

The logic of all pages follows “Check first, Act Later” design. This can be demonstrated on one of the pages: **prescribe.php** *(Figure 3).*

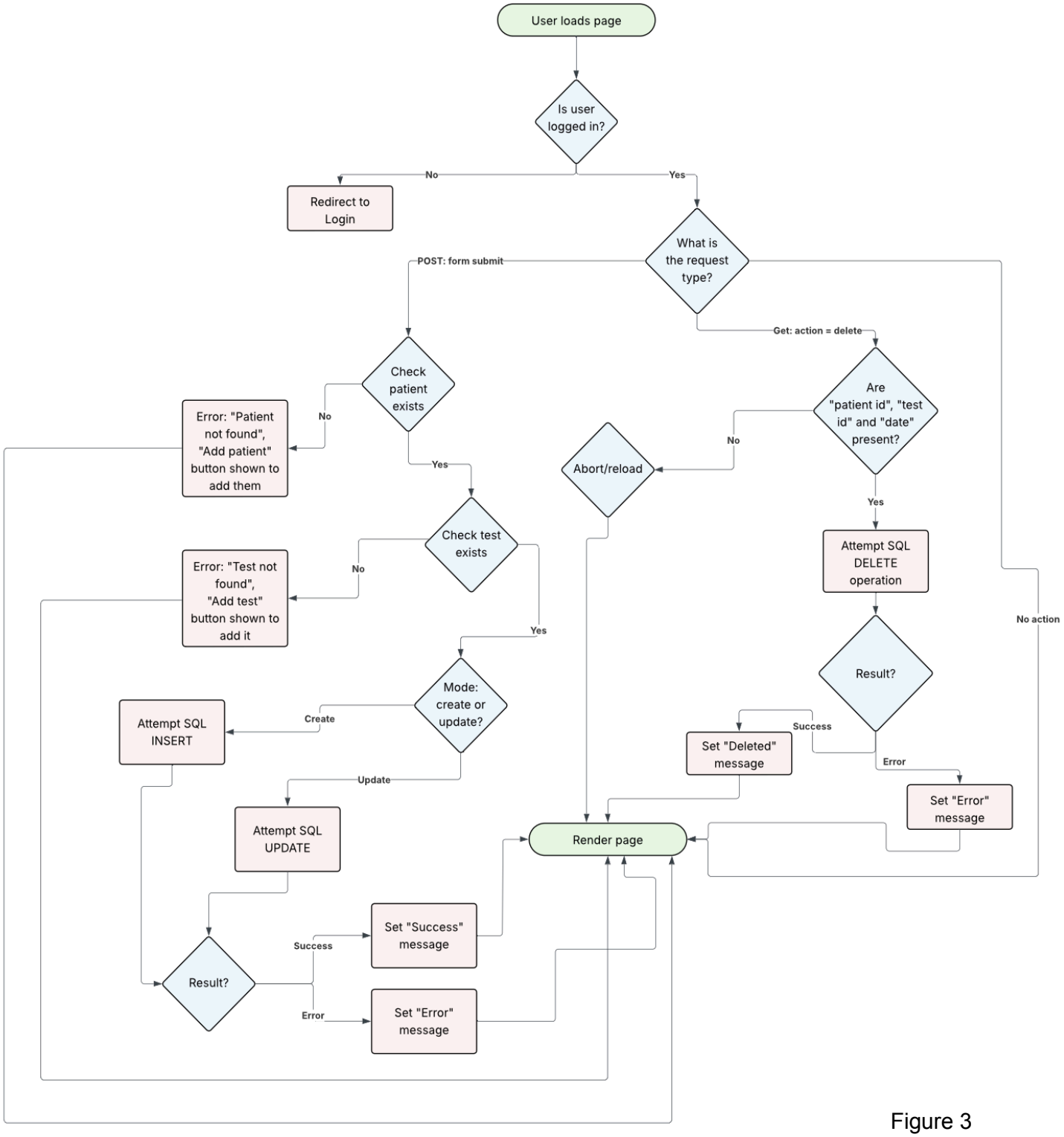

Figure 3

- The delete functionality: triggered by the "Delete" button, the system validates URL parameters (pid, tid, date) before attempting a SQL DELETE. The code catches specific database integrity errors to block the deletion of prescriptions linked to existing test results, preventing data corruption.
- The form submission flow: upon submission, the system checks if the patient and test exist, dynamically offering "Add" buttons if they are missing. If valid, the system executes an INSERT or UPDATE, while also catching possible database errors if they occur.

b. PHP Components

One of the key components that is present in all pages where CRUD functionality is possible — **executeAndLog() function**. It is defined in audit_trail.php *(Figure 4).*

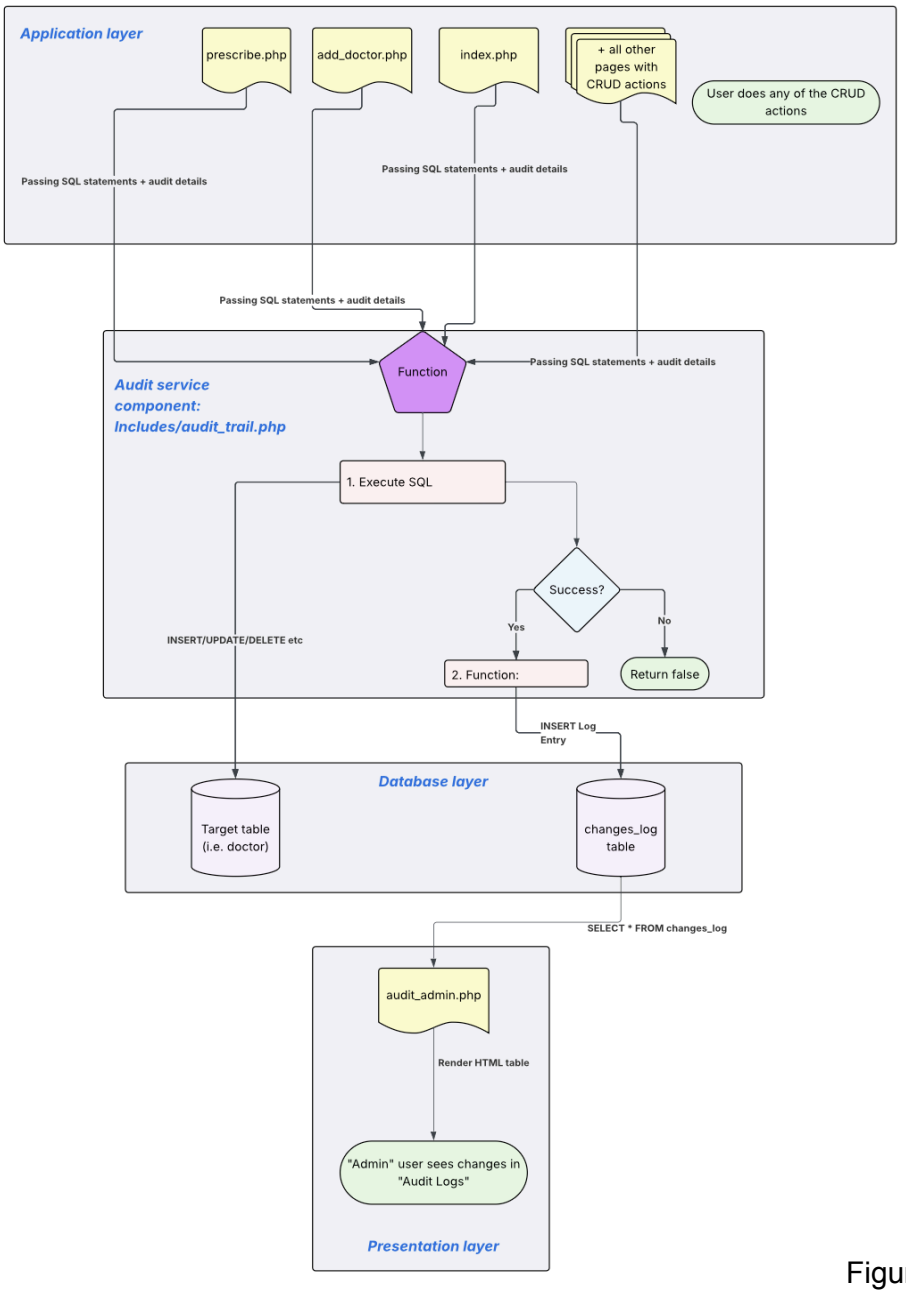

Figure 4

- Audit service (audit_trail.php): This component implements the executeAndLog() function to ensure data integrity. Application pages (e.g. prescribe.php) do not execute SQL directly. Instead, they pass the action to this function first.
- Admin audit logs (audit_admin.php): This page reads from the audit table (changes_log), transforming database rows into a human-readable interface for administrators.

4. Extending functionality

This platform can be further improved. The main priority should be tightening up security. Firstly, it is important to swap the current plain-text password storage for proper hashing using PHP's password_hash(). Secondly, while the current prepared statements handle some SQL injection risks, I'd strongly recommend adding stricter input validation.